暖屿插件开发手册

Warm Isle Plugin Developer Guide

从写一个包，到它真的在对话里被用出来

适用版本	暖屿插件商店 v2.3 / 宿主 1.0.0 / apiVersion 1
文档版本	v1.0 · 2026-11-10
读者	想给暖屿写能力插件的开发者、部署者
维护	暖屿 · 小满

目录

1 手册说明

2 十分钟快速开始

3 插件包结构

4 manifest 字段详解

5 四类 runtime

6 意图与触发

7 记忆

8 语音与媒体

9 权限与安装确认

10 发布与审核

11 测试与验收清单

12 常见错误与排查

13 安全红线

14 附录

15 唱歌插件全链路契约示例

提示：本手册按「先跑通、再写对、最后上架」的顺序组织。第一次写插件，建议从第 2 章直接开始。

1 手册说明

暖屿的插件不是编译进主程序的 Java 类，而是一个 zip 包：里面放 manifest、说明文档和素材，装上就能用。插件作者不需要拿到主程序源码，也不需要改一行 Java。

插件运行在四种 runtime 之一：`data`、`prompt`、`hosted`、`local`。前两种由宿主直接执行，后两种由作者或部署者自己提供服务，宿主通过 HTTP 调用。

1.1 商店是怎么运转的

整个生态只有一个官方商店（A 主机）。任何部署了暖屿代码的实例（B/C/D）都可以：

- 浏览 A 已审核上架的插件，点安装，从 A 下载 zip 并校验 sha256，装到本地；
- 在自己后台的「插件商店 → 上架插件」上传新包，包会进入 A 的待审队列；
- A 在后台的「上架审核」里通过后，B/C/D 刷新商店才能看到这个插件。

发布不需要注册账号，也没有作者后台。审核权只在 A：审核接口绑定 A 的内网地址和管理令牌，其他部署拿不到。

1.2 四类 runtime 怎么选

runtime	插件在哪跑	包里有没有代码	适合什么
data	宿主内查表	没有，只有素材	点歌、表情包、语录、固定素材
prompt	宿主代调模型	没有，只有模板+数据	塔罗、心理测试、写诗
hosted	作者的服务器	没有，只有 manifest	需要自己的算力、模型、数据库
local	部署者自己起服务	没有，只有安装说明	必须用本机 GPU / 软件 / 内网环境

> **先选简单的**
>
> 九成插件用 data 或 prompt 就够了。hosted 要长期维护域名、HTTPS、可用性和计费；local 要把安装步骤写清楚。能不用 hosted 就别用。

2 十分钟快速开始（data 型）

先做一个最小的「点歌」插件：包里有 2 段音频，用户说「唱首歌」时随机发一段。它不需要服务器、不需要模型调用，是最适合第一次上手的类型。

2.1 准备目录

```
menran.sing/
├── manifest.json
├── README.md
├── CHANGELOG.md
└── assets/
    ├── twinkle.wav
    └── birthday.wav
```

2.2 写 manifest

manifest.json（可直接复制修改）

```
{
  "schemaVersion": 2,
  "id": "menran.sing",
  "name": "唱歌",
  "version": "1.0.0",
  "apiVersion": 1,
  "runtime": "data",
  "description": "点歌就唱，没说歌名就随机唱一首",
  "author": { "name": "menran" },
  "license": "CC0-1.0",
  "tags": ["娱乐", "音乐"],
  "compat": { "minHostVersion": "1.0.0" },
  "intent": {
    "id": "music.sing",
    "examples": ["唱首歌", "唱个歌", "来一首", "想听你唱", "给我唱一段吧", "唱一个吧"],
    "exclusive": false
  },
  "actions": [{
    "name": "sing",
    "description": "为用户唱一首歌",
    "params": { "song": { "type": "string", "required": false, "description": "歌名" } },
    "returns": ["audio"],
    "timeoutMs": 15000
  }],
  "voice": { "strategy": "audio" },
  "catalog": {
    "key": "song",
    "entries": [
      { "value": "小星星", "aliases": ["小星星", "twinkle"],
        "file": "assets/twinkle.wav", "durationMs": 8000 },
      { "value": "生日快乐", "aliases": ["生日快乐", "birthday"],
        "file": "assets/birthday.wav", "durationMs": 7000 }
    ]
  }
}
```

2.3 写 README

README 会渲染在商店详情页，至少写清：插件做什么、说什么话会触发、会不会联网/写记忆、素材来源与许可证。

2.4 打包

把目录内容压成 zip，扩展名用 `.zip`。注意：压缩的是目录里的文件，不是外层目录本身。

```
cd menran.sing
zip -r ../menran.sing-1.0.0.zip .
```

> **包体限制**
>
> 实例后台上传走 Spring 的 30MB 限制，商店自身允许 50MB；解压后总大小 ≤120MB，单文件 ≤10MB，条目 ≤300。音频单文件 ≤1MB，图片 ≤5MB。

2.5 发布、审核、安装

1. 在 B 的后台打开「插件商店 → 上架插件」，选择 zip，点「上传并体检」。
2. 体检通过后进入 A 的待审队列；A 在「插件商店 → 上架审核」里点「通过」。
3. B/C/D 刷新商店，「已上架」列表出现这个插件；点「安装」→ 预览 → 确认。
4. 到「已安装」给某个机器人账号「启用」，第一次会弹出权限确认。

2.6 验证它真的会唱

在 QQ 里对机器人说「唱首歌」。预期：只收到一条语音，没有多余文字。再说「唱首不存在的歌」应收到一句友好提示，而不是报错刷屏。

> **看不到商店?**
>
> 先确认实例配置了 `PLUGIN_ENABLED=true`，并且能访问 `https://store.mziran.cn`。商店地址和公钥是写死的，不能改。

3 插件包结构

```
<id>-<version>.zip
├── manifest.json          必需，UTF-8，≤64KB
├── README.md              必需，商店详情页渲染
├── CHANGELOG.md           建议
├── icon.png               可选，256×256，≤200KB
├── preview.png            可选，1200×630，≤300KB
├── SETUP.md               仅 local 型：部署者安装/启动说明
├── assets/                可选，data 型素材
├── prompts/               可选，prompt 型模板
└── data/                  可选，prompt 型数据
```

3.1 允许与禁止

项	规则
允许的扩展名	.json .md .txt .csv .yaml .yml .wav .mp3 .ogg .m4a .flac .png .jpg .jpeg .webp .gif .svg .ttf .otf .woff .woff2
禁止	任何可执行文件（.sh .exe .dll .so .jar .class .py .js 等一律拒绝，local 型也不例外）
路径	必须是包内相对路径；禁止 `..`、绝对路径、符号链接、中文路径、加密 zip
编码	manifest / README / 模板统一 UTF-8

3.2 目录与文件命名

- 插件 `id` 用 `作者.能力`，发布后不可改，记忆 key 依赖它；
- 版本用严格 semver（`1.0.0`），已上架版本不能覆盖，改内容必须升版本号；
- 素材文件建议全小写、用连字符，避免跨平台解压乱码。

4 manifest 字段详解

manifest 是插件的身份证，也是宿主和商店做校验的唯一依据。下面按重要性排列。

4.1 顶层字段

字段	必填	规则 / 说明
schemaVersion	是	固定 2
id	是	`^[a-z0-9][a-z0-9._-]{2,48}$`，例如 `menran.sing`，发布后不可改
name	是	2–20 字，商店内不重名
version	是	严格 semver，如 `1.2.0`
apiVersion	是	固定 1；宿主不认识的版本会拒绝
runtime	是	`data` / `prompt` / `hosted` / `local`
description	是	≤80 字，进商店列表
author.name	是	署名；邮箱只在商店后台可见
license	是	SPDX 标识；缺失拒绝上架
tags	是	1–6 个
compat.minHostVersion	是	semver；低于此版本的实例看不到插件
intent	是	意图声明，见第 6 章
actions	是	至少 1 个，声明参数与返回类型
permissions	是	声明联网/写记忆/本机服务；缺省视为全不授权
memory	否	不写就等同 `policy: none`
quota	否	默认每人每天 5 次，上限 50
voice.strategy	音频必填	`audio` 或 `lyrics`；没有音频时不能写
hosted	hosted 必填	endpoint 必须 HTTPS，且域名在 permissions 里
local	local 必填	endpointHint / invoke / setup
prompt	prompt 必填	system 模板路径、maxTokens、temperature
catalog	data 必填	素材表；决定用户说到的名字怎么命中

4.1.1 最小 manifest 示例

只写必填字段也能跑起来；实际插件再按需要加 memory / voice / hosted 等。

```
{
  "schemaVersion": 2,
  "id": "menran.echo",
  "name": "回声",
  "version": "1.0.0",
  "apiVersion": 1,
  "runtime": "data",
  "description": "把一句固定的话回给你",
  "author": { "name": "menran" },
  "license": "CC0-1.0",
  "tags": ["测试"],
  "compat": { "minHostVersion": "1.0.0" },
  "intent": {
    "id": "menran.echo.say",
    "examples": ["说句话", "回我一句", "测试一下", "说点什么", "来一句"]
  },
  "actions": [{ "name": "say", "returns": ["text"] }],
  "catalog": {
    "key": "text",
    "entries": [{ "value": "hello", "file": "assets/hello.txt" }]
  }
}
```

4.2 actions 与参数

```
"actions": [{
  "name": "sing",
  "description": "为用户唱一首歌。用户没说歌名就随机。",
  "params": {
    "song":  { "type": "string", "required": false, "description": "歌名" },
    "style": { "type": "enum", "required": false,
               "values": ["温柔", "欢快", "伤感"] }
  },
  "returns": ["audio"],
  "timeoutMs": 60000
}]
```

- `name` 用 `^[a-z][a-z0-9_]{1,30}$`，插件内唯一；
- `returns` 决定宿主怎么发送：`audio` / `image` / `text` / `card`；
- `timeoutMs` 只影响同步调用，宿主硬上限 20 秒；超过请走异步；
- 必填参数缺失时，宿主会反问或让模型补参，不会瞎猜。

4.2.1 参数示例

```
用户说：唱首小星星，温柔一点
路由结果：intent=music.sing，action=sing
传给插件：{"song": "小星星", "style": "温柔"}
```

4.3 id、版本与兼容

`id` 是记忆、配额、吊销的键，一旦发布就不能改。改名等于做一个新插件。

`version` 是排他锁：同一个 id + version 已上架后，再次上传会被拒（`VERSION_FROZEN`），必须升版本。宿主只按 semver 比较，不认 `v1.0` 这种写法。

`compat.minHostVersion` 用于渐进升级：老实例不会在商店里看到装不上的插件。

5 四类 runtime

runtime 决定插件在哪执行、宿主怎么把结果交回对话。四类共用同一套 manifest、意图、记忆和权限模型，区别只在执行器。

5.1 data：查表 + 读素材

data 型没有任何可执行代码。宿主拿着用户这句话去 catalog 里找条目，找到后读包内文件，按文件类型发送。

```
"catalog": {
  "key": "song",
  "match": ["小星星", "生日快乐"],
  "entries": [
    { "value": "小星星", "aliases": ["小星星", "twinkle", "一闪一闪"],
      "file": "assets/twinkle.wav", "durationMs": 8000 },
    { "value": "生日快乐", "aliases": ["生日快乐", "birthday"],
      "file": "assets/birthday.wav", "durationMs": 7000 }
  ]
}
```

字段	说明
catalog.key	参数名，宿主用 `params[key]` 取用户说的名字；缺省 `item`
catalog.match	可选；用于路由命中的关键词/别名集合
entries[].value	条目的规范名；写进记忆的 `song` 变量就是它
entries[].aliases	用户可能说的其他叫法；大小写不敏感
entries[].file	包内相对路径，必须在插件目录内
entries[].durationMs	音频时长，用于 caption 与记忆（毫秒）

- 用户没说名字：宿主随机挑一条；
- 用户说了一个名字：先精确匹配 `value`，再匹配 `aliases`，最后做包含匹配；
- 都没找到：返回 `NOT_FOUND`，作者可以写 `userHint` 决定用户看到什么；
- 音频 ≤1MB、图片 ≤5MB；文本文件按 UTF-8 读出直接发。

> **data 型最安全**
>
> 包里没有可执行文件，也没有网络请求。商店审核最快，也最不容易出事。

5.1.1 命中示例

用户说	catalog 命中	结果
唱首小星星	value 精确匹配「小星星」	发 assets/twinkle.wav

来个生日快乐	aliases 匹配「生日快乐」	发 assets/birthday.wav
唱首歌	没给名字	随机挑一条
唱首七里香	没有这个条目	返回 NOT_FOUND + userHint

5.2 prompt：宿主代调模型

prompt 型适合「模板 + 数据 + 模型生成」的插件，比如塔罗、心理测试、写诗。插件自己不拿 API key，也不承担模型费用；宿主用部署者的模型额度代调。

```
{
  "runtime": "prompt",
  "prompt": {
    "system": "prompts/tarot.md",
    "maxTokens": 600,
    "temperature": 0.7
  },
  "data": {
    "cards": "data/cards.json"
  },
  "variables": ["date", "cards", "profile"],
  "actions": [{
    "name": "draw",
    "params": { "question": { "type": "string", "required": false } },
    "returns": ["text"]
  }]
}
```

项	规则
prompt.system	包内相对路径，只允许 .md / .txt，≤16KB
prompt.maxTokens	100–1000，缺省由宿主控制
prompt.temperature	0–2
data	键名 `^[a-z][a-z0-9_]{0,30}$`；值必须是包内 .json/.md/.txt/.csv；键会变成模板变量
variables	只能声明白名单内的变量：`date`、`profile`、action 参数名、data 键
模板渲染	只进 user 消息，不进 system；宿主自己做字面替换，不执行模板引擎

宿主默认提供的变量：

- `{date}`：今天日期（yyyy-MM-dd）；
- `{profile}`：从记忆库取的 3 条用户画像摘要；没有就写「还不太了解」；
- `{action 参数名}`：用户/模型补出来的参数；

- `{data 键}`：包内 data 文件的内容（JSON 会转成字符串）。

模板里出现改变身份的话术（如「忽略以上指令」「ignore previous」）会在上架体检时被拒；模板只进 user 消息，system guard 仍会拦截人格劫持。

> **prompt 型的边界**
>
> 插件只负责「一次触发、一次产出」。需要多轮交互、需要自己保存状态的，不要做成 prompt 型。

5.2.1 模板与渲染示例

```
prompts/tarot.md 的内容：

今天是 {date}。
关于对方：{profile}
ta 的问题是：{question}

请用三张牌给出解读，先给牌面，再给一句建议。
```

宿主用真实值做字面替换后，把它作为 **user 消息** 交给模型；system 仍然是宿主的固定护栏。

```
渲染后的 user 消息：

今天是 2026-11-10。
关于对方：喜欢周杰伦；最近在准备考试；讨厌被敷衍
ta 的问题是：这段关系还要继续吗？

请用三张牌给出解读，先给牌面，再给一句建议。
```

5.3 hosted：作者的服务器

hosted 型把执行放到作者自己的服务器，宿主通过 HTTPS 调用。适合需要自建算力、模型、数据库的插件。

```
"hosted": {
  "endpoint": "https://api.example.com/nuanyu/sing",
  "health": "/health",
  "auth": "Bearer optional-token"
}
```

`endpoint` 必须是 HTTPS，且域名要出现在 `permissions.network.outbound` 里；`health` 必填；`auth` 可选，会原样放进 `Authorization` 头。

5.3.1 请求

```
POST /v1/invoke
Content-Type: application/json
```

```
X-Plugin-Protocol: 1
X-Install-Id: 6f1c...（安装时随机生成的 UUID，与 QQ 号无映射）

{
  "apiVersion": 1,
  "invokeId": "9f2c1e6a-...",
  "pluginId": "menran.sing",
  "pluginVersion": "1.2.0",
  "action": "sing",
  "intentId": "music.sing",
  "params": { "song": "七里香" },
  "context": {
    "locale": "zh-CN",
    "timezone": "Asia/Shanghai",
    "time": "2026-11-10T10:00+08:00",
    "timeoutMs": 20000,
    "output": { "kinds": ["audio"], "maxBytes": 1048576 }
  }
}
```

宿主保证不会发送：QQ 号、昵称、群号、聊天原文、历史消息、精确到秒的调用时间。插件能拿到的只有业务参数、`installId` 和上面这些上下文。

5.3.1.1 原始 HTTP 示例

```
POST /v1/invoke HTTP/1.1
Host: api.example.com
Content-Type: application/json
X-Plugin-Protocol: 1
X-Install-Id: 6f1c2b7e-...

{"apiVersion":1,"invokeId":"9f2c...","pluginId":"menran.sing",
 "pluginVersion":"1.0.0","action":"sing","intentId":"music.sing",
 "params":{"song":"小星星"},"context":{"locale":"zh-CN",
 "timezone":"Asia/Shanghai","time":"2026-11-10T10:00+08:00",
 "timeoutMs":20000,"output":{"kinds":["audio"],"maxBytes":1048576}}}
```

5.3.2 同步响应

```
{
  "ok": true,
  "outputs": [
    { "kind": "audio", "mime": "audio/mpeg", "durationMs": 32000,
      "sha256": "...", "url": "https://cdn.example.com/a.mp3" }
  ],
  "caption": "唱了《七里香》",
  "memory": { "vars": { "song": "七里香", "duration": 32 } }
}
```

字段	规则
outputs[].kind	`audio` / `image` / `text`
outputs[].url	HTTPS；域名需在 permissions 里；宿主会做 SSRF 校验并限制 15s / 5MB

outputs[].dataBase64	小文件可直接内联；音频 ≤1MB，图片 ≤5MB
caption	必须描述「机器人做了什么」，不能写成用户的事实；宿主加全角括号
memory.vars	只允许回填模板变量；不能自带 key/value/importance（宿主统一渲染）

5.3.2.1 图片与文本输出示例

```
// 图片
{ "ok": true,
  "outputs": [{ "kind": "image", "mime": "image/png",
                "url": "https://cdn.example.com/cat.png" }],
  "caption": "发了一张猫咪图" }

// 纯文本
{ "ok": true,
  "outputs": [{ "kind": "text", "text": "今天适合休息，别硬撑。" }],
  "caption": "说了一句关心的话" }
```

5.3.3 异步响应（长任务）

同步调用硬上限 20 秒。唱歌、视频生成这类超过 20 秒的任务，立即返回 pending，宿主先发一句 ack，再轮询结果。

```
// 首次响应
{ "ok": true, "pending": true, "taskId": "t_123", "ackText": "好，等我一下" }

// 宿主每 2 秒 GET /v1/tasks/t_123，最长 90 秒
{ "ok": true, "outputs": [ ... ], "caption": "唱完了" }
```

5.3.3.1 异步任务完整示例

```
// 1）首次调用：立即返回，不阻塞
{ "ok": true, "pending": true, "taskId": "t_123",
  "ackText": "好，等我一下" }

// 2）宿主 GET /v1/tasks/t_123（每 2 秒，最长 90 秒）
{ "ok": true,
  "outputs": [{ "kind": "audio", "mime": "audio/mpeg",
                "durationMs": 32000,
                "url": "https://cdn.example.com/sing.mp3" }],
  "caption": "唱了《小星星》",
  "memory": { "vars": { "song": "小星星", "duration": 32 } } }
```

5.3.4 错误响应

```
{
  "ok": false,
  "error": {
```

```
    "code": "BAD_PARAMS",
    "message": "不支持的曲风",
    "userHint": "这首我唱不好，换一首？",
    "retryable": false
  }
}
```

code	含义	是否计熔断
BAD_PARAMS	参数不合法	否
NOT_FOUND	没有这个资源	否
UNAVAILABLE	依赖服务不可用	是
TIMEOUT	超时	是
RATE_LIMITED	作者侧限流	否
INTERNAL	作者代码异常	是
UNSUPPORTED	不支持该 action	否
OUTPUT_TOO_LARGE	结果超过体积	否

5.3.4.1 错误对应的 HTTP 状态

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{ "ok": false,
  "error": { "code": "BAD_PARAMS", "message": "不支持的曲风",
             "userHint": "这首我唱不好，换一首？", "retryable": false } }
```

5.3.5 硬性要求

- **幂等**：同一个 `invokeId` 重复调用必须返回同一结果，不能重复扣费/生成；
- **超时**：同步 ≤20 秒；更长请返回 pending；
- **体积**：响应体 ≤5MB；音频 ≤1MB；图片 ≤5MB；
- **跳转**：不允许 302 到未声明域名；
- **内容类型**：只认 audio/*、image/*、text/plain；不收 HTML/脚本；
- **元数据**：宿主会对音频/图片剥元数据后再发送。

5.3.6 接收端：你要自己起一个 HTTP 服务

hosted 没有 SDK，也不要求语言。你只需要在公网暴露一个 HTTPS 服务，实现两个路由：`GET /health` 和 `POST /v1/invoke`（异步再加一个 `GET /v1/tasks/{taskId}`）。宿主会按 manifest 的 endpoint 拼出完整 URL。建议 `/health` 额外返回 `version`，宿主要用它比对服务端版本。

最小 Node 示例（Express）：

```
import express from 'express'

const app = express()
app.use(express.json({ limit: '1mb' }))

// 幂等：同一个 invokeId 重复请求返回同一结果
const done = new Map()

app.get('/health', (_req, res) => res.json({ ok: true, version: '1.0.0' }))

app.post('/v1/invoke', (req, res) => {
  const { invokeId, action, params } = req.body

  if (done.has(invokeId)) return res.json(done.get(invokeId))

  if (action !== 'sing') {
    return res.status(400).json({
      ok: false,
      error: { code: 'UNSUPPORTED', message: 'unknown action',
               userHint: '这个我还没学会', retryable: false }
    })
  }

  // song 非必填：作者自己兜底
  const song = params.song || '小星星'
  const reply = {
    ok: true,
    outputs: [{
      kind: 'audio', mime: 'audio/mpeg', durationMs: 8000,
      url: 'https://cdn.example.com/sing/xiaoxingxing.mp3'
    }],
    caption: `唱了《${song}》`,
    memory: { vars: { song, duration: 8 } }
  }
  done.set(invokeId, reply)
  res.json(reply)
})

app.listen(34000, '127.0.0.1')
```

生产上不要自己处理证书，交给 Caddy / Nginx 终止 TLS，应用只监听 127.0.0.1：

```
api.example.com {
    handle /v1/* {
        reverse_proxy 127.0.0.1:34000
    }
    handle /health {
        reverse_proxy 127.0.0.1:34000
    }
}
```

manifest 里就写 `"endpoint": "https://api.example.com"`、`"health": "/health"`。宿主会请求 `https://api.example.com/v1/invoke`。

如果你的服务想挂在子路径（`https://api.example.com/nuanyu/sing`），要么在应用里挂载同样的前缀，要么在 Caddy 用 `handle_path` 把前缀剥掉再转发。

Python 也一样，FastAPI 十行就能接：

```
from fastapi import FastAPI, Request
app = FastAPI()


@app.get("/health")
def health():
    return {"ok": True, "version": "1.0.0"}


@app.post("/v1/invoke")
async def invoke(req: Request):
    body = await req.json()
    song = body.get("params", {}).get("song") or "小星星"
    return {
        "ok": True,
        "outputs": [{"kind": "audio", "mime": "audio/mpeg",
                     "durationMs": 8000,
                     "url": "https://cdn.example.com/sing/x.mp3"}],
        "caption": f"唱了《{song}》",
        "memory": {"vars": {"song": song, "duration": 8}},
    }
```

> **接收端的三条硬要求**
>
> HTTPS 有效；/health 3 秒内 200；同一个 invokeId 幂等。宿主不会重试一个已经成功的 invokeId 来重复扣费，但你必须自己防重。

5.4 local：部署者自建服务

local 型不是「宿主替你装软件」，而是部署者按你写的 `SETUP.md` 自己把服务跑起来，然后在后台填地址；宿主只做 HTTP 调用，不启动进程、不碰 docker.sock、不分配端口。

```
"local": {
  "endpointHint": "http://host.docker.internal:34001",
  "health": "/health",
  "invoke": "/v1/invoke",
  "setup": "SETUP.md",
  "requires": [
    { "name": "ffmpeg", "hint": "4.4+" },
    { "name": "NVIDIA Driver", "hint": "535+", "optional": true }
  ]
},
"permissions": {
  "localService": { "allow": true, "reason": "连接本机合成服务" },
  "memory": { "write": true }
}
```

`SETUP.md` 至少写清：依赖与版本、安装命令、启动命令与端口、如何验证 /health、如何升级/卸载、数据存在哪。

- 包内仍然不允许任何可执行文件；服务由部署者自己安装；
- 接口契约与 hosted 完全一致（`/health`、`/v1/invoke`、错误码、幂等）；
- 地址由部署者填写，宿主保存到实例级配置；manifest 里的 `endpointHint` 只是建议；
- 如果服务在宿主机、而宿主跑在容器里，部署者通常要填 `host.docker.internal` 或容器名。

> **本地服务不是沙箱**
>
> local 型会在部署者机器上运行第三方服务。安装确认会明确提示「会连接你本机启动的服务」，部署者必须自己判断是否信任。

5.4.1 完整 local 示例

```
{
  "schemaVersion": 2,
  "id": "menran.sing-local",
  "name": "唱歌（本地）",
  "version": "1.0.0",
  "apiVersion": 1,
  "runtime": "local",
  "description": "用你本机的 GPU 合成，音频不出内网",
  "author": { "name": "menran" },
  "license": "MIT",
  "tags": ["娱乐", "音乐", "本地"],
  "compat": { "minHostVersion": "1.0.0" },
  "intent": {
    "id": "music.sing",
    "examples": ["唱首歌", "来一首", "想听你唱", "来一段", "唱个歌呗"]
  },
  "actions": [{
    "name": "sing",
    "params": { "song": { "type": "string", "required": false } },
    "returns": ["audio"],
    "timeoutMs": 120000
  }],
  "voice": { "strategy": "audio" },
  "local": {
    "endpointHint": "http://host.docker.internal:34001",
    "health": "/health",
    "invoke": "/v1/invoke",
    "setup": "SETUP.md",
    "requires": [{ "name": "NVIDIA Driver", "hint": "535+", "optional": true }]
  },
  "permissions": {
    "localService": { "allow": true, "reason": "连接本机合成服务" },
    "memory": { "write": true }
  },
  "memory": {
    "policy": "episodic",
    "type": "relation",
    "keyTemplate": "唱过:{song}",
    "valueTemplate": "《{song}》，{duration}秒",
    "importance": 0.85
  }
}
```

配套 SETUP.md 示例：

```
# 本地唱歌服务

## 依赖
- ffmpeg 4.4+
- NVIDIA 驱动 535+（可选）

## 启动
docker run -d --name nuanyu-sing -p 34001:8080   ghcr.io/menran/nuanyu-sing:1.0.0

## 验证
curl http://127.0.0.1:34001/health
# {"ok":true}
```

```
## 在暖屿后台填写
http://host.docker.internal:34001
```

6 意图与触发

插件不是「用户说了关键词就执行」。宿主先判断这句话是不是在请求这件事，再决定走哪个插件、参数够不够。写错 examples，插件要么永远不触发，要么在闲聊里乱触发。

6.1 examples 怎么写

`intent.examples` 是触发的唯一依据，5–10 条，必须是用户真的会说的完整句子。

要求	反例	正例
至少 3 个字	歌、唱	唱首歌、来一首
完整句子	音乐	放点音乐吧
不带具体实体名	唱七里香	唱个歌
覆盖不同说法	四条都是「唱歌」	唱首歌、来一段、想听你唱
不要泛词/语气词	看、来、那个、嗯	来一段、给我讲个故事

规则细节：

- 5–10 条，少于 5 条或多于 10 条都会被体检拒绝；
- 3 字示例默认只做「全等匹配」，但当它以请求动词开头（来/唱/放/给/帮/查/算/讲/画/提醒）时，允许做包含匹配；
- 含「他/她」的句子不会用 3 字示例做包含匹配，避免「他昨天唱首歌就走了」误触发；
- 示例里不要写歌名、地名、人名，这些应该靠参数抽取，而不是写死在意图里。

6.2 三档匹配

级别	规则	例子	成本
L1 全等	去掉首尾空白后与示例完全相同	「唱首歌」	0
L2 包含	句子包含 ≥4 字示例；或包含请求动词开头的 3 字示例（句子 ≤20 字、无他/她）	「你能给我唱首歌吗」	0
L3 语义	向量相似度 ≥0.80；只在前两档都没命中时触发；	「来一段周杰伦的」	一次 embedding

	有请求语气前置过滤		

L3 复用记忆模块的向量服务（bge-small-zh）；服务不可用时自动退回 L1/L2，不会阻塞对话。

6.3 情绪保护

命中情绪词（累、烦、难受、想哭、崩溃、压力、失眠……）且没有点单动词（唱、来一首、放、查、讲、画、提醒我）时，任何插件都不会触发。这是写死在宿主里的，作者无法关闭。

> **最容易翻车的地方**
>
> 用户说「今天好累啊」，机器人回「要不要我给你唱首歌」非常败好感。情绪保护就是为这句话存在的。

6.4 参数与反问

当前实现只做确定性抽取：catalog 精确/别名/包含 → enum 值 → 参数正则（如果声明了）。没有“模型补参”这一步；缺必填参数时宿主直接反问用户，不会把原句交给模型，更不会交给插件。

如果原句包含「上次 / 那首 / 刚才 / 再来 / 又」，宿主会查该用户最近 7 天的插件记忆，把实体名回填到缺失参数；这类追问不会再走语义相似度。

6.5 独占与共存

`intent.exclusive` 只该用于有独占副作用的意图（提醒、游戏、视频）。唱歌、塔罗、天气这类娱乐意图应允许多个插件共存，由采纳率和冷却仲裁；同一账号启用第二个独占插件时会被拒绝。

6.6 触发示例

用户说	命中级别	结果
唱首歌	L1 全等	直接执行，不调模型
你能不能唱首歌啊	L2 包含（3 字请求句）	直接执行
来一段周杰伦的	L2 包含「来一段」	直接执行，随机唱一首
今天好累啊	情绪保护	不触发，正常聊天
你会唱歌吗	能力问句	只回答「会」，不执行

7 记忆

插件默认不写记忆。只有 manifest 声明了 `memory.policy` 且 `permissions.memory.write=true`，宿主才会在结果发送成功后写一条。

7.1 policy

policy	行为	适用
none（默认）	不写记忆	塔罗、查天气、一次性查询
episodic	写一条「我做过的事」	唱歌、发视频、讲故事
commitment	当前按 episodic 写入，不自动安排到期提醒	暂不建议依赖；提醒能力后续再接

7.2 模板

```
"memory": {
  "policy": "episodic",
  "type": "relation",
  "keyTemplate": "唱过:{song}",
  "valueTemplate": "《{song}》，{duration}秒",
  "importance": 0.85
}
```

运行期插件只需要回填变量，不要自带 key/value：

```
"memory": { "vars": { "song": "七里香", "duration": 32 } }
```

7.3 宿主强制的硬约束

- `subject` 只能是 `bot`，不会污染用户画像；
- `type` 只能 `relation` 或 `event`；

- `importance` 封顶 0.85；
- `value` 渲染后 6–80 字，太短或太长都不写；
- 每天每个插件每个用户最多 5 条；
- key 会自动加插件前缀（`menran.sing:唱过:七里香`），两个插件写同一个 key 不会互相覆盖；
- `source` 记为 `plugin:<id>`，卸载时按前缀归档，不物理删除。

> **key 必须带实体**
>
> 只写 `唱过` 的话，机器人永远只记得最后一首。把 `{song}` 放进 key，才能回答「上次那首」。

7.4 写入结果示例

```
memories 表新增一行:

qq_id      = 100000000
type       = relation
subject    = bot
key        = menran.sing:唱过:小星星
value      = 《小星星》，8 秒
source     = plugin:menran.sing
importance = 0.85
```

8 语音与媒体

8.1 音频

只要 action 的 `returns` 包含 `audio`，manifest 就必须声明 `voice.strategy`。

值	含义	当前支持
audio	插件返回成品音频	支持；宿主校验、剥元数据、发送
lyrics	只给歌词/曲调，宿主用机器人音色合成	协议预留，尚未接入；不要依赖

- 音频 ≤1MB；建议 ≤60 秒，否则协议端转码和发送都会变慢；
- 宿主会先发再落历史；发送失败不会写记忆；
- 插件语音与日常语音共用频控：90 分钟间隔、每天 3 条；明确点播最多 6 条；
- 不要把 base64 写进 caption，也不要让插件直接碰 OneBot。

8.2 图片

图片 ≤5MB。宿主发送前会过 NSFW 预筛，敏感图直接丢弃并降级为文字。图片同样会剥 EXIF 后再发。

8.3 caption 的写法

正确	错误
「唱了《七里香》」	「你喜欢七里香」
「发了一张猫咪表情包」	「你想要一只猫」

caption 必须描述机器人做了什么。写成用户的事实，会被记忆模块当成用户画像。

8.4 媒体输出示例

```
// 音频：voice.strategy = audio
{ "ok": true,
  "outputs": [{ "kind": "audio", "mime": "audio/mpeg",
                "durationMs": 32000,
                "url": "https://cdn.example.com/a.mp3" }],
  "caption": "唱了《小星星》" }

// 图片：宿主会先过 NSFW 预筛
{ "ok": true,
  "outputs": [{ "kind": "image", "mime": "image/png",
                "url": "https://cdn.example.com/a.png" }],
  "caption": "发了一张图" }
```

9 权限与安装确认

权限不是装饰，是安装/启用前的二次确认依据。宿主只相信 manifest 声明的权限，但会加上自己的硬限制。

权限	manifest 写法	确认时机	用户看到什么
联网	`permissions.network.outbound: ["api.example.com"]`	安装 + 启用	会把参数发送到 api.example.com，不发送聊天内容
写记忆	`permissions.memory.write: true`	安装	会往记忆库写记录，每天最多 5 条
本机服务	`permissions.localService: {allow:true, reason:"..."}`	安装 + 启用	会连接你在本机启动的服务
模型额度	prompt 型自动产生	启用	会调用你的模型，单次约 N tokens

- 权限摘要只算权限字段，作者改描述不会让用户重新确认；
- 升级/回滚后若权限变化，会要求重新确认；
- 隐私模式下，hosted / local 不允许启用；
- 没声明 `permissions.memory.write` 却写 `memory.policy`，体检直接拒。

9.1 权限声明示例

```
{
  "permissions": {
    "network": {
      "outbound": ["api.menran.cn"],
      "purpose": "合成歌曲音频"
    },
    "memory": { "write": true },
    "localService": null
  }
}
```

9.2 quota 示例

```
{
  "quota": { "perUserPerDay": 10, "globalPerHour": 60 }
}
```

10 发布与审核

暖屿没有作者账号。任何部署了本项目的实例，登录自己的后台就能发布；包统一进 A 的待审队列，A 审核通过后所有实例可见。

10.1 发布流程

1. B 后台 →「插件商店」→「上架插件」→ 选 zip →「上传并体检」。
2. 实例服务端把包转发到官方商店的 `/api/v1/publish/upload`。
3. 商店做结构体检与 manifest 语义校验；通过后写 `pending`。
4. A 后台 →「插件商店」→「上架审核」→ 看 manifest/README/来源 → 通过或驳回。
5. 通过后重建索引；B/C/D 刷新商店即可看到并安装。

10.2 状态与版本

状态	含义	对已装实例
pending	已提交，等待审核	不可见
approved	已上架	商店可见、可安装
rejected	被驳回	不可见；修改后可重传同版本
yanked	作者/管理员下架	已装的继续可用，商店标「已下架」
blocked	拉黑/吊销	停止触发（验签生效后），不自动卸载

版本规则：同一个 `id + version` 已上架后不能覆盖，必须升版本号；被驳回/待审的版本可以覆盖。

10.3 限流与容量

项	限制
发布频率	单 IP 5 次/小时、20 次/天
待审队列	最多 100 条；满了会提示稍后再试
包体	实例后台上传 30MB（Spring 限制）；商店自身允许 50MB
解压	总大小 ≤120MB、单文件 ≤10MB、条目 ≤300

10.4 签名与密钥轮换

商店的 `index.json` 和吊销名单都用 Ed25519 签名，实例内置官方公钥验签。商店被入侵但没有私钥也投不了毒。

A 要换签名密钥时，在服务器执行 `scripts/rotate-key.mjs`，用旧私钥签一条新公钥记录并追加到轮换链；实例下次拉索引发现 keyId 不认识，会自动验链、采纳并落盘，不需要发版。

> **轮换的前提**
>
> 旧私钥仍可信。如果旧私钥已经泄漏，应改用离线根密钥方案，而不是用旧私钥继续签轮换。

10.5 发布与审核接口示例

```
# 1) 任意部署实例发布（不需要账号，包进 pending）
curl -X POST https://store.mziran.cn/api/v1/publish/upload   -H "X-Publisher: my-instance"
-F "file=@menran.sing-1.0.1.zip"

{"ok":true,"submissionId":12,"pluginId":"menran.sing",
 "version":"1.0.1","sha256":"8f2d...","status":"pending"}

# 2) A 的实例后台读取待审（走自己的服务端，不直接暴露 admin）
GET /api/store/admin/pending
{"ok":true,"items":[{"submissionId":12,"pluginId":"menran.sing",
  "version":"1.0.1","publisher":"my-instance",
  "submittedAt":"2026-11-10T07:00:00.000Z"}]}

# 3) A 通过
POST /api/store/admin/review/12
{"action":"approve","note":""}
{"ok":true}
```

10.6 商店索引与下载示例

```
# index.json（片段）
{"schemaVersion":1,"generatedAt":"2026-11-10T07:00:00Z",
 "plugins":[{"id":"menran.sing","name":"唱歌","latest":"1.0.1",
   "runtime":"data","apiVersion":1,"minHostVersion":"1.0.0",
   "intent":"music.sing","sha256":"8f2d...","size":460825}]}

# 下载
GET /api/v1/download/menran.sing/1.0.1
X-Checksum-Sha256: 8f2d...
Content-Type: application/zip
```

10.7 密钥轮换链示例

```
GET /api/v1/keys/rotation

{"schemaVersion":1,"currentKeyId":"7c1e...",
 "chain":[
   {"from":"eacda113a48371ab","to":"7c1e...",
    "newPublicKey":"MCowBQYDK2Vw...","effectiveAt":"2026-11-10T07:00:00Z",
    "signature":"base64(旧私钥对前四个字段的签名)"}
 ]}
```

10.8 更新可见性与服务版本巡检

store 只知道 manifest 版本；hosted 的代码在作者服务器上，local 的代码在部署者机器上。为了不让“代码已经变了、后台还显示旧版本”，宿主会定期巡检。

状态	含义	后台显示
ok	服务端 version 与本地 manifest 一	不额外提示

	致	
outdated	服务端 version 不一致	警告「服务端 1.0.3」
unreachable	健康检查失败/超时	「服务不可达」
unconfigured	local 还没填端点	「未配置地址」
unknown	健康检查 200 但没返回 version	「服务未报版本」

巡检频率：启动 1 分钟后一次，之后每 30 分钟一次；后台「检查服务版本」可以手动触发。

- **hosted**：版本不一致说明作者服务端更新了。B 不需要重新下载代码，但应该去商店看有没有新 manifest；作者必须升版本并重新审核，否则行为变更绕过了审核。
- **local**：版本不一致说明 B 的镜像旧了。后台只做提示，不会自动 `docker pull`；B 按 SETUP 更新镜像并重启服务。
- 巡检**不阻断触发**，只做可见性：服务端可能故意灰度，不能因为版本号不同就停掉。

更新说明：包里的 `CHANGELOG.md` 会随版本存进商店，后台升级弹窗会显示「更新说明」。

首页导航的「插件商店」会显示可升级数量，前端每 15 分钟刷新一次商店索引；打开插件页也会立即刷新。

> **作者更新代码的正确姿势**
>
> data/prompt：升版本 → 上传 → A 审核 → B 点升级。hosted：先升 manifest 版本并过审，再发布服务端代码；local：升 manifest 版本并更新镜像 tag，SETUP 里写清升级命令。

11 测试与验收清单

上架前至少把下面这些跑一遍。商店的自动体检只覆盖结构与字段，行为要你自己测。

11.1 通用

#	检查项	通过标准
1	manifest 能被解析	上传体检无硬错误
2	意图示例	说 2–3 条示例能触发；说无关闲聊不触发
3	情绪保护	说「今天好累」不触发
4	参数缺失	反问一句，不执行、不写记忆
5	结果发送	用户只收到一条结果，没有多余文字
6	历史与记忆	历史里有一条「唱了…」；记忆 key

		带实体名
7	发送失败	断网/停协议端时，不写记忆，只回一句提示
8	重复调用	同一 invokeId 不重复生成/扣费

11.2 hosted / local 额外

- `/health` 3 秒内返回 200；
- 每个 action 用默认参数调一次，返回 `ok`；
- 故意传错参数，返回 `BAD_PARAMS` 而不是 500；
- 停掉服务，返回 `UNAVAILABLE`，宿主能熔断；
- 音频 ≤1MB、图片 ≤5MB；
- 60 秒内返回，或正确走 `pending` + `taskId`；
- 不依赖请求里的 QQ 号/昵称（根本拿不到）。

11.3 后台试跑

实例后台提供 `POST /api/plugins/{id}/try`，可以不走聊天、直接跑一次 action，返回 `kind`、`caption`、`sizeBytes`、`durationMs`、`memoryVars`。上架前建议先用它确认素材和参数没问题。

11.4 试跑接口示例

```
POST /api/plugins/menran.sing/try
{"action":"sing","params":{"song":"小星星"}}

{"ok":true,"kind":"audio","caption":null,"sizeBytes":352844,
 "durationMs":8000,"memoryVars":{"song":"小星星","duration":"8"}}
```

12 常见错误与排查

错误	原因	怎么修
EXAMPLES_COUNT	不是 5–10 条	补齐到 5 条以上，删掉重复说法
EXAMPLE_TOO_SHORT	示例少于 3 字	改成完整请求句，如「唱首歌」
MISSING_ACTIONS	没有 actions	至少声明 1 个 action 和 returns
MISSING_VOICE	返回 audio 但没写 voice.strategy	加 `"voice": {"strategy":"audio"}`
INSECURE_ENDPOINT	hosted 用了 http	换 HTTPS
DOMAIN_NOT_DECLARED	endpoint 域名不在 outbound	把域名加进 permissions.network.outbound
MISSING_ENDPOINT	hosted 没写 endpoint	补 endpoint 和 health
LOCAL_NOT_DECLARED	local 没声明本机服务权限	permissions.localService 设为 true 或 {allow:true}
MISSING_TEMPLATE	prompt 没写 system	补 `.md`/`.txt` 模板路径
PROMPT_INJECTION	模板含注入话术	删掉「忽略以上指令」这类句子
VERSION_FROZEN	同版本重复上传	升版本号再传
PUBLISH_LIMITED	发布太频繁	等一小时或换网络
QUEUE_FULL	待审队列满	联系商店所有者处理积压
OUTPUT_TOO_LARGE	结果超体积	压缩音频/图片，或改成分段
TIMEOUT	同步超过 20 秒	改异步：返回 pending + taskId
UNAVAILABLE	服务连不上/5xx	检查域名、证书、进程

12.1 错误响应长什么样

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{"ok":false,"errors":[
  {"field":"manifest.intent.examples[2]","code":"EXAMPLE_TOO_SHORT",
   "message":"示例句至少 3 个字"}
]}
```

13 安全红线

下面每一条都写死在宿主或商店校验里，不是建议。

1. **插件不能碰 OneBot。** 插件只能返回内容，发不发、怎么发由宿主决定。能直接调 OneBot / 协议端的插件可以群发、加好友、@全体，封的是部署者的号。
2. **不要试图拿 QQ 号、昵称、群号、聊天原文。** 宿主构造请求体，不传这些字段；manifest 也没法声明要它们。
3. **包里不允许任何可执行文件。** data / prompt / hosted / local 都一样；local 的服务由部署者自己安装。
4. **不要运行期远程改行为。** 从自己服务器下发脚本、配置来改变已审核行为，发现即拉黑。
5. **不要做隐藏遥测。** 不收集实例数据、不上报用户标识；需要统计就在商店侧做匿名下载量。
6. **素材要有版权。** 歌曲、图片、字体必须自制/授权/公版；商店展示 license，收到投诉先下架。
7. **不要把密钥写进包。** manifest 和 README 都是公开的；需要鉴权的 hosted 服务，auth 只放部署者配置。

> **责任边界**
>
> 做了中心分发，商店所有者就从「作者」变成了「分发者」。所以上架必须人工审核，下架/吊销必须随时可用。

14 附录

14.1 完整 hosted manifest 示例

```
{
  "schemaVersion": 2,
  "id": "menran.sing",
  "name": "唱歌",
  "version": "1.0.0",
  "apiVersion": 1,
  "runtime": "hosted",
  "description": "点歌就唱，支持温柔/欢快/伤感",
  "author": { "name": "menran", "url": "https://mziran.cn" },
  "license": "MIT",
  "tags": ["娱乐", "音乐"],
  "compat": { "minHostVersion": "1.0.0" },
  "intent": {
    "id": "music.sing",
    "examples": ["唱首歌", "来一首", "想听你唱", "来一段", "唱个歌呗"],
    "exclusive": false
  },
  "actions": [{
    "name": "sing",
    "description": "为用户唱一首歌。没说歌名就随机。",
    "params": { "song": { "type": "string", "required": false } },
    "returns": ["audio"],
    "timeoutMs": 60000
  }],
  "voice": { "strategy": "audio" },
  "hosted": {
    "endpoint": "https://api.menran.cn/nuanyu/sing",
    "health": "/health",
    "auth": ""
  },
  "permissions": {
    "network": { "outbound": ["api.menran.cn"], "purpose": "合成歌曲音频" },
    "memory": { "write": true }
  },
  "memory": {
    "policy": "episodic",
    "type": "relation",
    "keyTemplate": "唱过:{song}",
    "valueTemplate": "《{song}》，{duration}秒",
    "importance": 0.85
  },
  "quota": { "perUserPerDay": 10, "globalPerHour": 60 }
}
```

14.2 local 的 permissions 写法

```
"permissions": {
  "localService": { "allow": true, "reason": "连接本机合成服务" },
  "memory": { "write": true }
}
```

商店体检同时接受 `"localService": true` 和对象写法；推荐对象写法，因为 reason 会显示给部署者。

14.3 术语表

术语	含义
runtime	插件执行方式：data / prompt / hosted / local
intent	意图，用户「想干什么」的抽象，不是关键词
examples	意图的示例句，路由的唯一依据
exclusive	独占意图，同一账号只能启用一个实现
installId	安装时随机生成的 UUID，发给插件用于标识安装，与 QQ 号无映射
熔断	连续失败后暂停调用，避免拖垮主链路
短路	插件处理完直接返回，不再走模型回复
keyring	实例保存的商店公钥集合，支持密钥轮换

14.4 版本记录

版本	日期	说明
v1.0	2026-11-10	首版：四类 runtime、意图、记忆、语音、发布审核、测试清单

15 一句话的契约：用户说「我要听歌」

这一章只跟一句话：用户在 QQ 里说「我要听歌」。我们看这句话在每一个契约里长什么样，尤其是 HTTPS：插件到底收到原始句子，还是收到抽好的参数？

> **先给结论**
>
> 插件永远拿不到「我要听歌」这句原文。宿主只把结构化 params 和少量上下文发给插件；当前实现没有“模型抽参数”这一步，缺必填参数时宿主直接反问用户，不会调用插件。

15.1 第一站：OneBot 事件

B 主机上的 qqbot 从 OneBot 收到的是这样一条私聊事件（简化）：

```
{
  "post_type": "message",
  "message_type": "private",
```

```
  "self_id": 100000000,
  "user_id": 100000000,
  "raw_message": "我要听歌",
  "time": 1794294000
}
```

MessageDispatcher 先过话题屏蔽、情绪观察，然后在媒体计划之前把这句话交给插件网关。到这一步，原始句子仍然只在宿主进程里。

15.2 第二站：路由契约

唱歌包 1.0.2 的 examples 里有「我要听歌」，所以这是最直接的 L1 全等命中：

```
输入：我要听歌
命中：music.sing
级别：L1 全等
分数：1.0
action：sing
params：{}          ← 句子里没有具体歌名
```

用户说	命中方式	params	谁决定唱哪首
我要听歌	L1 全等「我要听歌」	{}	插件/宿主随机
我想听歌	L2 包含「想听歌」	{}	插件/宿主随机
我要听歌，小星星	L2 包含「我要听歌」	{"song":"小星星"}	指定小星星
今天好累啊	情绪保护	—	不触发，正常聊天

说明：1.0.1 的 examples 里没有「我要听歌」，语义相似度也只有 0.70，所以那句话当时不会命中；1.0.2 补了示例，才成为 L1。

15.3 第三站：参数抽取契约（关键）

宿主拿到路由结果后，做的是**确定性抽取**，不调用模型：

1. 查 catalog：用户这句话里有没有条目的 `value` 或 `aliases`；
2. 匹配 enum 参数值；
3. 如果参数声明了正则，按正则抽；
4. 必填参数仍缺失 → 返回 `FILL_PARAMS`，让宿主反问一句，**不调用插件**。

```
「我要听歌」          → params = {}
「我要听歌，小星星」  → params = {"song": "小星星"}
「我要听歌，七里香」  → params = {}      ← catalog 里没有七里香，不会瞎填
```

所以，对「模型抽参数还是直接把原始请求给 HTTPS」这个问题，答案是：

问题	当前实现
模型参与抽参数吗?	不参与。路由和参数抽取都是宿主本地的确定性逻辑。
原始句子会发给 HTTPS 插件吗?	不会。请求体里没有 raw_text / message / history 字段。
插件拿到什么?	params（可能为空）+ context + installId。
参数不够怎么办?	宿主反问用户，例如「想听哪首呀？」，不调用插件。

> **以后如果加模型补参**
>
> 也只会把抽出来的**值**放进 params，不会把原句一起发出去；这是出站字段白名单，不是作者能改的配置。

15.4 第四站：HTTPS 契约（hosted 版）

如果唱歌插件是 hosted 型，宿主对「我要听歌」发出的请求是：

```
POST https://api.menran.cn/nuanyu/sing/v1/invoke
Content-Type: application/json
X-Plugin-Protocol: 1
X-Install-Id: 6f1c2b7e-...

{
  "apiVersion": 1,
  "invokeId": "9f2c1e6a-...",
  "pluginId": "menran.sing",
  "pluginVersion": "1.0.2",
  "action": "sing",
  "intentId": "music.sing",
  "params": {},
  "context": {
    "locale": "zh-CN",
    "timezone": "Asia/Shanghai",
    "time": "2026-11-10T16:00+08:00",
    "timeoutMs": 20000,
    "output": { "kinds": ["audio"], "maxBytes": 1048576 }
  }
}
```

- 请求体里没有「我要听歌」四个字，也没有 QQ 号、昵称、历史；
- `params` 是空的，因为这句话没给歌名；
- 插件作者要自己决定随机唱哪首，或者返回 `BAD_PARAMS` 让宿主反问。

作者服务器可以这样回：

```
{
  "ok": true,
  "outputs": [{ "kind": "audio", "mime": "audio/mpeg",
                "durationMs": 32000,
                "url": "https://cdn.menran.cn/sing/random.mp3" }],
  "caption": "唱了《小星星》",
  "memory": { "vars": { "song": "小星星", "duration": 32 } }
}
```

如果作者把 song 声明成 required: true，流程会变成：

```
「我要听歌」→ 路由命中 → song 必填但缺失
→ FILL_PARAMS：宿主回「想听哪首呀？」（不请求 HTTPS）
→ 用户说「小星星」→ params 补齐 → 才发 HTTPS 请求
```

15.5 同一句话，data 版收到什么

线上这个唱歌包是 data 型，没有 HTTPS。宿主自己在本地完成：

```
路由：music.sing / sing
params：{}
catalog：没指定歌名 → 随机三选一
选中：生日快乐 → assets/voice/birthday.wav（308,740 字节，7 秒）
输出：kind=audio, durationMs=7000,
      memoryVars={song:"生日快乐", duration:"7"}
```

如果用户说的是「我要听歌，小星星」，params 就是 {"song":"小星星"}，会精确命中 twinkle.wav。

15.6 同一句话，local 版收到什么

local 与 hosted 的请求体完全一样，只是地址由部署者填写：

```
POST http://host.docker.internal:34001/v1/invoke
（请求头、params、context 与 15.4 完全相同）
```

15.7 异步契约：如果这首歌要合成 30 秒

```
// 首次：立即确认，不阻塞用户队列
{ "ok": true, "pending": true, "taskId": "t_123",
  "ackText": "好，等我一下" }

// 宿主每 2 秒轮询 GET /v1/tasks/t_123，最长 90 秒
{ "ok": true, "outputs": [{ "kind":"audio", "url":"https://.../a.mp3" }],
  "caption": "唱了《小星星》" }
```

15.8 发送、历史与短路

```
宿主发送：sendPrivateRecordSync(100000000, <bytes>, 10000) → true

messages 新增：
  role=assistant  content=（唱了《生日快乐》，7 秒）  kind=voice

然后 return true：不再调用 ChatService.reply，模型不会参与这一轮。
```

15.9 记忆与运行记录

```
memories:
  key=menran.sing:唱过:生日快乐
  value=《生日快乐》，给 ta 唱了一遍
  source=plugin:menran.sing
  importance=0.8  subject=bot  type=relation

plugin_runs:
  plugin_id=menran.sing  plugin_version=1.0.2
  action=sing  status=ok  latency_ms=1266
```

15.10 如果这句话没命中

比如「我要睡觉」不在意图里，也不含请求线索：插件网关返回 false，正常走模型对话。用户不会收到语音，也不会写插件记忆。

情绪保护同理：说「今天好累」时即使 examples 里有「我要听歌」也不会触发，因为情绪词优先。

15.11 一句话的数据边界总表

数据	会不会到插件	说明
原始句子「我要听歌」	不会	请求体没有 raw_text/message 字段
抽出来的 params	会	可能为空对象
installId	会	随机 UUID，与 QQ 号无映射
locale / timezone / time	会	时间截到分钟
QQ 号 / 昵称 / 群号	不会	宿主构造请求体，字段白名单写死
聊天历史	不会	插件拿不到上下文
模型生成的参数	当前不会	没有模型抽参；缺参只反问

发布、安装、审核这些商店侧的契约不在这条请求路径上，见第 10 章和第 15 章之外的 10.5—10.7。